SDC HACKATHON 2026

Technical & Tools Declaration

Hosted by Xiamen University Malaysia (XMUM) AI Club
Submission Deadline: 25 October 2026

Instructions: Declare every API, cloud service, AI tool, dataset, model, and sponsor resource your team used to build the project. Complete every section; where a category does not apply, tick "No" or write "None" – do not leave it blank. Submit as a single PDF document named "SDC2026_TeamName_TechDeclaration.pdf" together with your project submission. Undeclared third-party resources discovered at judging may affect your score or eligibility.

1. Team & Project Overview

Team Name	We got no idea
Project / Solution Name	AgriGuard
Name (Leader)	Tang Yun Tian
Email & Phone (Leader)	yuntian1003@gmail.com 0124098320
University Affiliation(s)	Universiti Sains Malaysia
Live Prototype URL	https://xmum-hack.vercel.app/
Code Repository URL (public / access granted to judges)	https://github.com/yuntian1003/XMUM_HACK.git

2. Architecture & Technology Stack

2.1 System Architecture Overview

Describe how the components declared in this document connect end-to-end (e.g., user interface → backend → AI model/API → database → cloud hosting). Attach a system block diagram as an appendix if available.

[Maximum 200 words]

AgriGuard is a full-stack TypeScript web application. The React 19 / Vite frontend is deployed on Vercel encompassing Farm Desk, Farmer Hub, and the Government Dashboard and communicates with a Node.js / Express backend deployed in a Docker container on Render via type-safe tRPC. For crop scans, an on-device TensorFlow.js MobileNet model delivers instant offline leaf verification and preliminary disease detection. When online, the backend invokes a Google Genkit pipeline that sends the image to Gemini 3.1 Flash Lite, grounded by Vertex AI Search (RAG) and validated against Pl@ntNet for botanical identification with Groq Llama vision fallback.. Scan imagery is stored in AWS S3, while scans, users, zones, stock inventory, and activity logs are persisted in PostgreSQL (Supabase). In Farm Desk, a worker captures a leaf photo, a manager reviews AI recommendations and checks chemical inventory, and a WhatsApp-ready draft order is prepared for owner approval.

2.2 Languages, Frameworks & Libraries

List the main programming languages, frameworks, and third-party libraries that your team did not create. Open-source use is permitted provided it is declared and properly licensed/attributed.

Name & Version	Purpose in Project	License
TypeScript 5.9.3	Core language (full-stack frontend + backend)	Apache-2.0
Node.js 20+	Backend server runtime	MIT
React 19.2	Frontend component UI library	MIT
Vite 7.1	Frontend development server & production bundler	MIT
Tailwind CSS 4.1	Utility-first application styling	MIT
Radix UI / shadcn/ui	Accessible UI primitives and dialogs	MIT

Name & Version	Purpose in Project	License
Wouter 3.3	Lightweight client-side routing	ISC
tRPC 11.6 + TanStack React Query 5.9	End-to-end type-safe client-server communication	MIT
Framer Motion 12.2	UI transitions & micro-animations	MIT
Recharts 2.15	Interactive charts & data visualisation	MIT
TensorFlow.js 4.22 + MobileNet 2.1.1	On-device Edge AI leaf & disease screening	Apache-2.0
Express 4.21	Backend HTTP and API server	MIT
Drizzle ORM 0.44 + Drizzle Kit 0.31	Database schema management & type-safe queries	Apache-2.0
Google Genkit 1.32	Structured AI workflow orchestration	Apache-2.0
Vitest 2.1	Automated unit and integration testing	MIT
esbuild 0.25	High-performance backend bundling	MIT
pnpm 10.4, Prettier 3.6	Package management and code formatting	MIT

3. APIs & Cloud Services

3.1 External APIs

List every external API your solution calls (GenAI, maps, payments, messaging, data feeds, etc.). Do not include API keys or secrets in this document.

API & Provider	Purpose in Project	Access Tier (free / paid / sponsor-provided)
Gemini API via Genkit (Google)	Multimodal crop disease diagnosis and advisory generation	Free / Developer
Groq API	High-speed vision fallback for crop diagnosis	Free
Vertex AI Search (Google Cloud)	RAG agronomy retrieval for treatment guidance	Free / Cloud trial
Pl@ntNet API	Botanical plant species and organ identification	Free
FoodMonitor Data Feeds	Commodity market prices, FX, fuel, and trade indices	Free

3.2 Cloud Platforms & Deployment

List every cloud service used for hosting, storage, databases, authentication, or compute, and indicate how the prototype is deployed.

Service & Provider	Purpose in Project	Plan / Region
Vercel	Hosts the React / Vite web frontend	Free Hobby / Singapore
Render	Runs the backend API service inside a Docker container	Free / Singapore
PostgreSQL (Supabase)	Relational database (scans, inventory, users, logs)	Free Serverless / Singapore
AWS S3	High-resolution crop scan image storage	Free / Singapore
Google Cloud Platform	Vertex AI Search for agronomic RAG knowledge	Free Trial / Singapore

4. AI Tools, Models & Datasets

4.1 AI Models Used in the Solution

List every AI model embedded in or called by your application, including pre-trained, fine-tuned, and self-trained models.

Model & Version	Source / Provider	Usage (pre-trained / fine-tuned / self-trained)	License / Terms
MobileNet v1 (2.1.1)	TensorFlow.js models	Pre-trained, client-side inference	Apache-2.0
Gemini-3.1-flash-lite-preview	Google	Pre-trained, called via Genkit API	Google API Terms
Llama-4-scout-17b-16e-instruct	Groq / Meta	Pre-trained, vision fallback API	Llama 4 Community License

4.2 Datasets

List all datasets used for training, fine-tuning, evaluation, or in the running application. State clearly if a dataset contains personal or sensitive data.

Dataset Name	Source / URL	Type (public / private / self-collected / synthetic)	License	Personal Data? (Y/N)
Synthetic paddy-farm data *(zones, SOPs, stock, suppliers, users)*	Team-created	Synthetic	Team-owned	N
ASEAN Crop & Treatment Knowledge Base	Curated agronomy literature (17 ASEAN crops)	Team-compiled	Team-owned	N
World Bank & Food Commodity Feeds	https://datacatalog.worldbank.org/	Public	CC BY 4.0	N

4.3 AI-Assisted Development Tools

Tools used to help write code, design, content, or documentation (e.g., chat assistants, code completion tools, image generators). Declaring these tools does not reduce your score; undeclared use may.

Did your team use any generative-AI tool to help create code, design, text, or other project material?

☑ Yes ☐ No

If yes, list them below:

Tool & Version	**Used For**	**Extent of Use (minor / moderate / major)**
ChatGPT (GPT-4o)	documentation, proposal drafting	minor
Claude (Claude-3.7-Sonnet)	code, debugging, documentation, proposal drafting	minor
Gemini (Gemini 2.5 Pro /Flash & Antigravity IDE)	code, debugging, documentation, proposal drafting	minor
Replit	documentation, proposal drafting, demo and pitching flow	minor

Briefly describe how your team reviewed, tested, and took responsibility for AI-generated output:

[Maximum 100 words]

All AI-generated code and text was read, run and edited by team members before being committed. We ran the TypeScript type-check and Vitest tests, and manually tested the full Farm Desk journey (upload, manager review, stock check, owner approval, activity log) on synthetic data. AI-generated suggestions were not accepted without review, and secrets were kept out of the repository. The team leader takes responsibility for the final code, written content and all declarations in this document.

5. Sponsor-Provided Resources

Declare any official SDC sponsor-provided GenAI API, cloud credits, tools, mentoring, or data that your team used.

Did your project use any official SDC sponsor-provided resource?

☒ Yes ☐ No

If yes, list them below:

Sponsor	**Resource (API / credits / platform / data)**	**How It Was Used in the Project**
Replit	credits	We use it for drafting ideas, pitching and demo flow.

Was the sponsor resource essential to the core functionality of the prototype, or could it be replaced? Explain briefly:

[Maximum 100 words]

It was used strictly as an ideation and collaborative drafting workspace during early brainstorming and demo planning. The deployed AgriGuard application is fully decoupled, containerised with Docker, and self-hosted across standard cloud platforms (Vercel, Render, AWS S3, and Supabase). The prototype runs independently on standard developer environments using local TypeScript toolchains and does not depend on Replit at build or runtime.

6. Security, Privacy & Responsible Use

6.1 Data Handling & Privacy

What user or personal data does your application collect, store, or send to third-party services (including AI APIs)? How is it protected, and how long is it kept? State "No user data is collected" if applicable.

[Maximum 150 words]

The prototype uses synthetic farm records; no real farmer personal data is collected. The system manages user accounts (name, email, role via JWT session cookies), uploaded crop photos (AWS S3), and scan history, stock, and audit logs (PostgreSQL on supabase). Offline captures persist in local browser IndexedDB before syncing to the cloud. Crop images are transmitted via HTTPS to Google Gemini and Pl@ntNet for diagnosis. Client-side TensorFlow.js scans execute entirely on-device without data transfer. Secrets and credentials (JWT_SECRET, database URLs, API keys) reside in environment variables. Suppliers only receive order line items without farm yield data. Prototype data is retained only for the hackathon evaluation period. Production deployment will comply with Malaysia's Personal Data Protection Act (PDPA), enforcing TLS 1.3 in transit, AES-256 at rest, and automated data deletion policies.

6.2 Known Limitations & Risks

Identify the main technical limitations of your AI components (e.g., hallucination, bias, rate limits, dependency on a single provider) and any mitigation you have implemented.

[Maximum 150 words]

- Misclassification: MobileNet and Gemini can misdiagnose under glare, motion blur, or overlapping foliage. The on-device model provides only initial screening. Mitigation: Managers review and can override diagnoses; scans flag confidence scores and escalate anomalies for human inspection.
- Hallucination: LLMs may generate improper chemical dosages. Mitigation: Genkit grounds outputs with Vertex AI Search (RAG) and curated ASEAN crop treatment tables; disclaimers explicitly state that recommendations do not replace certified agronomists.
- Provider Dependency & Rate Limits: Relying on external APIs (Gemini, Pl@ntNet, Vertex AI Search) risks downtime or quotas. Mitigation: Automatic failover to Groq (Llama-4 vision) if Gemini fails, offline Edge AI screening when disconnected, and mock fallbacks so the app operates without external services configured.
- Synthetic Data: Operational workflows and financial models have not yet undergone full longitudinal field testing in varied real-world monsoon microclimates.

7. Compliance & Readiness Checklist

Tick all boxes that apply before submitting.

- ☒ All APIs, cloud services, AI tools, models, and datasets used in the project are declared in this document.
- ☒ All third-party code, datasets, and models are used in accordance with their licenses, and attribution is provided where required.
- ☒ No API keys, passwords, or other credentials are exposed in the code repository, prototype, or this document.
- ☒ Any use of generative-AI tools for development has been disclosed and reviewed by the team.
- ☒ Any use of official SDC sponsor resources has been declared in Section 5.
- ☒ The prototype described here is accessible to judges through the links provided in Section 1.
- ☒ All fields in this template have been completed within the word limits stated.

8. Declaration

By submitting this document, the team leader confirms that the information provided is complete and accurate to the best of the team's knowledge, and that every API, cloud service, AI tool, dataset, model, and sponsor resource used in the project has been declared.



23/10/2026